

Documentación con Comentarios – Cypress + TypeScript + SauceDemo

Autor: Andres David Pereira

Proyecto: Automatización de pruebas E2E con Cypress

Página bajo prueba: <https://www.saucedemo.com/v1/>

1 Objetivo

Automatizar pruebas sobre la página **SauceDemo** implementando:

- **Comandos personalizados** (`cy.login`)
 - **Fixtures** para manejar credenciales
 - **Page Objects** para separar lógica de negocio
 - **TypeScript** para tipado y mejor mantenimiento
-

2 Estructura del proyecto

```
CYPRESS/  
├── cypress.config.ts    # Configuración de Cypress  
├── tsconfig.json       # Configuración de TypeScript  
├──  
├── cypress/  
│   ├── e2e/           # Casos de prueba  
│   │   ├── login.cy.ts  
│   │   └── products.cy.ts  
│   ├──  
│   ├── fixtures/      # Datos de prueba  
│   │   └── users.json  
│   ├──  
│   ├── pages/         # Page Objects  
│   │   └── LoginPage.ts
```

```
|   └── ProductsPage.ts
|
|   └── support/          # Configuración extra
|       ├── commands.ts  # Definición de comandos personalizados
|       ├── index.d.ts   # Tipado de comandos personalizados
|       └── e2e.ts        # Punto de entrada para cargar comandos
```

3 Configuración

cypress.config.ts

```
import { defineConfig } from 'cypress'

export default defineConfig({
  e2e: {
    baseUrl: 'https://www.saucedemo.com', // URL base del sitio a probar
    supportFile: 'cypress/support/e2e.ts', // Archivo de soporte
    specPattern: 'cypress/e2e/**/*.cy.ts' // Patrón de búsqueda de tests
  },
})
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "es6",
    "lib": ["es6", "dom"],
    "types": ["cypress"], // Se agregan tipos de Cypress
    "module": "esnext",
    "moduleResolution": "node",
    "strict": true,
    "esModuleInterop": true,
    "include": ["cypress/**/*.ts", "cypress/**/*.d.ts"]
  }
}
```

```
}
```

4 Comandos personalizados

[cypress/support/commands.ts](#)

```
// Definimos un comando personalizado llamado "login"
// que permite iniciar sesión con credenciales
Cypress.Commands.add('login', (username: string, password: string) => {
  cy.visit('/v1/') // Abrir la página de login
  cy.get('#user-name').type(username) // Escribir usuario
  cy.get('#password').type(password) // Escribir contraseña
  cy.get('#login-button').click() // Hacer click en login
})
```

Tipado de comandos personalizados

[cypress/support/index.d.ts](#)

```
/// <reference types="cypress" />

// Extendemos la interfaz de Cypress para que reconozca nuestro comando "login"
declare namespace Cypress {
  interface Chainable {
    /**
     * Inicia sesión en SauceDemo con usuario y contraseña
     * @example cy.login('standard_user', 'secret_sauce')
     */
    login(username: string, password: string): Chainable<void>
  }
}
```

Cargar los comandos

cypress/support/e2e.ts

```
// Aquí se importan todos los comandos personalizados
import './commands'
```

5 Page Objects

Separan los **selectores y acciones** de cada página.

LoginPage.ts

```
// Page Object para la página de login
export class LoginPage {
  // Abre la página de login
  visit() {
    cy.visit('/v1/')
  }

  // Método para iniciar sesión
  loginAs(username: string, password: string) {
    this.visit()
    cy.get('#user-name').type(username)
    cy.get('#password').type(password)
    cy.get('#login-button').click()
  }

  // Retorna el mensaje de error si ocurre un fallo de login
  getErrorMessage() {
    return cy.get('[data-test="error"]')
  }
}
```

ProductsPage.ts

```
// Page Object para la página de productos
export class ProductsPage {
```

```
// Retorna el título de la página
getTitle() {
  return cy.get('.title')
}

// Agrega al carrito un producto específico por su nombre
addProductToCartByName(name: string) {
  cy.contains('.inventory_item', name).find('button').click()
}

// Hace clic en el ícono del carrito
goToCart() {
  cy.get('.shopping_cart_link').click()
}
}
```

6 Fixtures

`cypress/fixtures/users.json`

```
{
  "users": {
    "standard": {
      "username": "standard_user",
      "password": "secret_sauce"
    },
    "locked": {
      "username": "locked_out_user",
      "password": "secret_sauce"
    }
  }
}
```

💡 Con esto evitamos poner credenciales "hardcodeadas" en los tests.

7 Casos de prueba

login.cy.ts

```
import { LoginPage } from '../pages/LoginPage'
import { ProductsPage } from '../pages/ProductsPage'

describe('Login Tests - SauceDemo', () => {
  const loginPage = new LoginPage()
  const productsPage = new ProductsPage()

  it('✅ Login exitoso con usuario válido', () => {
    cy.fixture('users').then((data) => {
      // Se toma usuario estándar desde fixtures
      loginPage.loginAs(data.users.standard.username, data.users.standard.
password)

      // Validación: se debe mostrar el título "Products"
      productsPage.getTitle().should('contain', 'Products')
    })
  })

  it('❌ Login fallido con usuario bloqueado', () => {
    cy.fixture('users').then((data) => {
      // Se usa usuario bloqueado
      loginPage.loginAs(data.users.locked.username, data.users.locked.pass
word)

      // Validación: debe aparecer mensaje de error
      loginPage.getErrorMessage().should('contain', 'locked out')
    })
  })
})
```

products.cy.ts

```

import { LoginPage } from '../pages/LoginPage'
import { ProductsPage } from '../pages/ProductsPage'

describe('Products Tests - SauceDemo', () => {
  const loginPage = new LoginPage()
  const productsPage = new ProductsPage()

  // Antes de cada test, iniciar sesión con usuario estándar
  beforeEach(() => {
    cy.fixture('users').then((data) => {
      loginPage.loginAs(data.users.standard.username, data.users.standard.
password)
    })
  })

  it('🛒 Agregar producto al carrito', () => {
    // Agregar mochila al carrito
    productsPage.addProductToCartByName('Sauce Labs Backpack')

    // Navegar al carrito
    productsPage.goToCart()

    // Validaciones
    cy.url().should('include', '/cart.html')
    cy.contains('Sauce Labs Backpack').should('be.visible')
  })
})

```

8 Ejecución de pruebas

Abrir el runner interactivo:

```
npx run cy
```

Ejecutar en modo headless (sin UI):

```
npx cypress run
```

10 Conclusiones

- Cypress + TS + Page Objects → estructura ordenada y escalable.
- `cy.login` → simplifica pruebas de login en múltiples specs.
- Fixtures permiten centralizar credenciales.
- Los comentarios facilitan el entendimiento para otros QA/Developers.